ChatBridge Pre-Implementation Research Document

Case Study Analysis

The TutorMeAI case study presents a deceptively simple premise (let third-party apps live inside a chat) that quickly reveals itself as a distributed systems problem. The core tension is between openness and control. The platform must support apps ranging from a simple calculator to a complex chess game with ongoing state, yet remain restrictive enough that a malicious or broken app cannot expose student data or deliver harmful content to children.

The first key problem is the **trust boundary between chat and third-party app**. In a K-12 context, this is not merely technical but ethical. Students are minors. Teachers are accountable. An app rendering UI inside the chat has been granted the platform's implicit endorsement. If it displays inappropriate content, collects unauthorized data, or crashes, the platform bears responsibility. We addressed this with defense in depth: apps run in sandboxed iframes with no parent DOM access, communication flows through a typed postMessage protocol with origin validation, and apps must be registered and approved before appearing in a classroom.

The second problem is **bidirectional state synchronization**. A chess game is the canonical example: the student says "let's play," a board appears, they make moves, ask the AI for help mid-game, and the AI must understand the current board state to respond. The chat cannot simply fire-and-forget tool invocations. It must maintain live awareness of what the app is doing. We solved this with a dual communication architecture. Tool invocations flow through the backend (LLM calls a tool, backend routes to the app, result feeds back). Real-time UI state flows directly between the iframe and frontend via postMessage, which the frontend includes as context in subsequent LLM calls. This keeps the backend stateless while giving the LLM rich context about the student's experience.

The third problem is **completion signaling**: knowing when an app interaction is "done" so conversation can resume naturally. We defined an explicit protocol: apps send an `app:complete` message with a summary, the frontend dismisses the iframe and injects the summary into conversation history, and the LLM references it in subsequent turns. A heartbeat mechanism detects unresponsive apps and triggers graceful recovery.

A significant trade-off was whether to build on Chatbox's existing MCP (Model Context Protocol) system or design a custom plugin architecture. MCP provides tool discovery and invocation but has no concept of UI rendering or completion signaling. A fully custom system offers complete control but means rewriting working infrastructure. We landed on a hybrid: a plugin adapter interface supporting three integration patterns (iframe+postMessage, MCP, REST+OAuth), allowing each app to connect via whatever protocol fits its nature. The same platform handles a custom-built

chess game, a cloud-hosted encyclopedia via MCP, and a Notion integration via OAuth, demonstrating genuine architectural flexibility.

The ethical dimension extends beyond content safety to data minimization (apps receive only the context they need, not the full conversation), explicit and revocable OAuth consent flows, and the power dynamic in a teacher-controlled tool marketplace. These concerns shaped the API contract from the start.

Phase 1: Define Your Constraints

1. Scale & Load Profile

Metric	Value	Rationale
Users at launch	1-5 concurrent	Evaluation/demo context
Users at 6 months	N/A (sprint project)	Production projections provided in cost analysis
Traffic pattern	Spiky	Student usage clusters around school hours and homework time
Concurrent app sessions per user	1	One active iframe at a time; previous apps collapsed into cards
Cold start tolerance for app loading	<2s	iframe apps are lightweight static React builds
Chat first-token latency	<3s	Standard LLM streaming expectation
postMessage round-trip	<50ms	Local browser communication, no network hop
Tool invocation end-to-end	<5s	Includes LLM function call decision + adapter routing + response

2. Budget & Cost Ceiling

Category	Budget	Notes
LLM API (development)	~$5	OpenRouter credits already available
LLM API (production)	Pay-per-use	~$0.003/1K input tokens, ~$0.015/1K output tokens via OpenRouter
Infrastructure	$0-20/mo	Railway free tier + Cloudflare Workers free tier
Auth service	$0	better-auth is self-hosted, no SaaS fees
Database	$0-5/mo	Railway Postgres free tier for demo
Total development spend	~$5	Nearly all cost is LLM tokens

LLM cost per tool invocation: ~$0.01-0.03 depending on context length. Average tool call adds ~500 input tokens (tool schemas) + ~200 output tokens (function call response). Acceptable for demo; at scale, would optimize with model routing (cheaper models for simple tool dispatch).

Where we trade money for time: Managed services everywhere. Railway for Postgres (vs self-hosted), Cloudflare Workers for MCP hosting (vs managing a server), better-auth (vs custom JWT implementation). The only "build it ourselves" decision is the Elysia backend, where the custom logic (plugin routing, OAuth vault) justifies it.

3. Time to Ship

Milestone	Deadline	Focus
MVP + Presearch	Tuesday, April 1	This document + architecture video
Early Submission	Friday, April 4	Full plugin system + 3+ apps working
Final Submission	Sunday, April 6	Polish, auth flows, docs, deployment

Speed-to-market vs maintainability: Speed-to-market for the sprint, but the plugin adapter interface is designed for long-term extensibility. The `PluginAdapter` contract is the one thing we

over-engineer because every app depends on it. Everything else (UI polish, admin features) is MVP-quality.

Iteration cadence after launch: Not applicable for this sprint. The architecture supports post-sprint iteration through the plugin system: new apps can be added without modifying core platform code.

4. Security & Sandboxing

Concern	Decision
Third-party app isolation	Sandboxed iframes: `sandbox="allow-scripts allow-forms"`, no `allow-same-origin` for untrusted apps
Malicious app registration	Apps must be pre-registered in the `app_registrations` table; no self-service registration in MVP
Content Security Policy	`frame-src` restricted to registered app domains only; `default-src 'self'` for platform
Data privacy between apps	Apps receive only their tool invocation params, never the full conversation or other apps' state
API key protection	OpenRouter key stored server-side only; never sent to browser
OAuth token security	Encrypted at rest (AES-256-GCM) in Postgres; transmitted only over HTTPS
Prompt injection via app state	App state sanitized, length-limited (max 2KB), and type-checked before LLM context injection
Rate limiting	Per-user limits on chat API (60 req/min); per-app limits on tool invocations (30 req/min)

5. Team & Skill Constraints

Team composition: Solo developer.

Skill/Technology	Level	Impact on Architecture
React/TypeScript	Expert	Can modify Chatbox's React codebase confidently
Zustand / State management	Expert	Leverage Chatbox's existing stores
Bun / Elysia	Expert	Backend framework chosen for familiarity
OAuth2 flows	Comfortable	Can implement Notion OAuth without research overhead
PostgreSQL	Expert	Drizzle ORM chosen for type-safety + familiarity
WebSockets / SSE	Expert	SSE for chat streaming is straightforward
LLM function calling	Expert	Tool schema design and dynamic injection
iframe / postMessage	New	Trivial API surface; biggest risk is protocol design, not implementation
Electron / Chatbox codebase	New	Mitigated by focusing on web-only; reading Platform/RequestAdapter seams first

Biggest skill gap: Chatbox codebase familiarity. Mitigation: spend first 2 hours mapping the data flow (message send to response render) before writing any code. The web platform abstraction (`WebPlatform`) and `RequestAdapter` interface are the key integration points.

Phase 2: Architecture Discovery

6. Plugin Architecture

Decision: iframe-based with postMessage protocol.

Approach	Pros	Cons	Verdict
iframe + postMessage	True security isolation, apps control their own UI, proven browser pattern	Cross-origin limitations, requires explicit communication protocol	**Selected**
Web Components / Shadow DOM	Style encapsulation, no cross-origin complexity	No JavaScript isolation (security risk for untrusted apps), framework coupling	Rejected
Server-side rendering	Full server control, no client-side security concerns	Apps can't have rich interactive UI, high latency	Rejected

How apps register tool schemas: Apps are pre-registered in the `app_registrations` Postgres table with a `config` jsonb column containing their tool schemas (name, description, parameters as JSON Schema). For MCP apps, schemas are discovered dynamically via the MCP `listTools()` protocol method.

Message passing protocol: postMessage with typed messages and origin validation. See full protocol definition in [Plugin Architecture details](#).

Runtime tool discovery: On each chat request, the backend queries enabled apps, gathers their tool schemas (from DB for iframe/REST apps, from MCP `listTools()` for MCP apps), and injects them as the `tools` parameter in the OpenRouter function calling request. The LLM sees only currently active tools.

7. LLM & Function Calling

Provider: OpenRouter (existing credits), primary model Claude Sonnet 4 for strong tool-use performance.

Concern	Decision
Function calling format	OpenAI-compatible format via OpenRouter (all major models support this)
Dynamic tool schema injection	Tool schemas gathered from active adapters per-request and passed as `tools` array
Context window management	Sliding window of last 20 messages + app state summaries. Tool schemas from 4 apps add ~2K tokens. Claude Sonnet 4 has 200K context, so headroom is large. If more apps are added, tool schemas would be filtered by relevance (only include apps the user has interacted with recently).
Streaming while waiting for tool results	LLM response streams via SSE until a `tool_call` is emitted. Stream pauses while the backend executes the tool. Tool result is fed back to the LLM, which resumes streaming a natural-language response. The frontend shows a "thinking" indicator during tool execution.
Model fallback	GPT-4o-mini via OpenRouter for cost optimization on simple routing queries (stretch goal)

8. Real-Time Communication

Channel	Technology	Direction	Purpose
Chat streaming	SSE (Server-Sent Events)	Server → Client	LLM response tokens, tool call notifications, completion events
App UI state	postMessage	iframe ↔ Frontend	Real-time state sync (board position, card progress), completion signals
API calls	HTTPS (REST)	Client → Server	Message submission, conversation CRUD, app management, OAuth flows

Why SSE over WebSocket for chat: SSE is simpler (unidirectional, auto-reconnect, works through proxies/CDNs), and chat streaming is inherently server-to-client. The client-to-server direction uses standard HTTP POST. Chatbox's existing architecture uses this pattern. WebSocket would add complexity with no benefit since we don't need server-initiated pushes outside of streaming responses.

Why postMessage over WebSocket for app communication: iframe apps are same-browser, so postMessage is zero-latency with no network hop. WebSocket would require each iframe app to maintain a separate server connection, adding infrastructure complexity. postMessage keeps the communication local and simple.

Reconnection and message ordering: SSE has built-in reconnection via `Last-Event-ID`. Each SSE event includes an incrementing ID. On reconnect, the client resends the last ID and the server replays missed events. For postMessage, ordering is guaranteed by the browser's single-threaded event loop. Tool invocations use unique IDs to match requests with responses, preventing out-of-order confusion.

9. State Management

State Type	Location	Persistence	Rationale
Chat state (conversations, messages)	PostgreSQL via Elysia API	Permanent	Must survive browser close, device switch
UI state (active panel, input draft)	Zustand (frontend, in-memory)	Session only	Ephemeral, cheap to recreate
App iframe state (board position, card progress)	Inside the iframe app (localStorage or in-memory)	App-controlled	Apps own their state; platform only sees `state:update` snapshots
App context for LLM	Zustand `appContext` map (frontend)	Session only	Rebuilt from postMessage `state:update` events; lost on page refresh
Auth state (session, tokens)	better-auth cookie + Postgres	Permanent	Session cookie auto-sent with requests
OAuth tokens (Notion)	PostgreSQL (encrypted)	Permanent	Must persist across sessions for seamless re-auth

How app context merges back into conversation history: When an app sends `state:update`, the frontend stores it in a Zustand `appContextStore` keyed by appId. On the next user message, the frontend includes this map in the `/api/chat/stream` request body. The backend injects it as a system message:

```
Active app 'chess' state: { fen: "rnbqkbnr/...", turn: "black", moveHistory: ["e4", "
```

When an app sends `app:complete`, the summary is persisted as a message with `role: 'system'` in the conversation, so it survives page refresh and is available in future LLM context.

State persistence across page refreshes: Chat history is in Postgres, so it survives. Active iframe state is lost (iframe is unmounted). On return, the app can be re-initialized, and the last `app:complete` summary or the last tool invocation result in the message history provides context for the LLM to resume naturally.

What happens to app state if the user closes the chat: The iframe is destroyed. For apps like Chess, the game state stored in the app's localStorage persists if the user returns to the same browser. The conversation history retains all tool invocation results and state summaries, so the LLM can reference what happened even if the app state itself is gone. For Flashcards, deck data persists via the app's own storage; review session progress is lost but the LLM can reference the last session summary.

10. Authentication Architecture

Platform auth vs per-app auth: Two separate layers.

Layer	Technology	Scope
Platform auth	better-auth (cookie-based sessions)	User identity for the chat platform. Required to use ChatBridge.
Per-app auth	OAuth2 (per-app, managed by backend)	Optional. Only Notion requires it. Chess, Flashcards, and Grokipedia require no user auth.

Token storage and refresh strategy: - Platform session tokens: managed by better-auth in `sessions` table. HttpOnly secure cookies. Automatic expiry and refresh handled by better-auth middleware. - OAuth tokens (Notion): stored in `user_app_tokens` table, encrypted at rest with AES-256-GCM. The encryption key is an environment variable, never committed to code. Refresh tokens are used to obtain new access tokens before they expire. The backend checks `expires_at` before each API call and refreshes proactively.

OAuth redirect handling within iframe context: OAuth does NOT happen inside an iframe (many providers block this via `X-Frame-Options`). Instead: 1. User clicks "Connect Notion" in the chat UI 2. Backend returns an authorization URL 3. Frontend opens the URL in a **new tab/popup** (not iframe) 4. User authorizes on Notion's site 5. Notion redirects to our callback URL (`/api/apps/notion/oauth/callback`) 6. Backend exchanges code for tokens, stores them, and renders a "success, you can close this tab" page 7. Original tab polls `/api/apps/notion/oauth/status` and updates the UI when connected

How auth requirements surface to the user: When the LLM tries to invoke a Notion tool and the user has no stored token, the backend returns a `tool_result` with `{ status: "auth_required", authUrl: "..." }`. The LLM receives this and responds naturally: "I need to connect to your Notion account first. Click here to authorize." The auth URL is rendered as a button in the chat message.

11. Database & Persistence

Schema design: See [full schema]() below.

How tool invocation history is stored: Every tool call is recorded in `tool_invocations` with: app_id, tool_name, input params, output result, status (pending/success/error/timeout), and duration_ms. This powers both the LLM context (tool results are stored as messages) and observability (query invocation success rates, latencies).

Read/write patterns and indexing:

Table	Primary Access Pattern	Index Strategy
conversations	List by user_id, ordered by updated_at	`idx_conversations_user_updated (user_id, updated_at DESC)`
messages	List by conversation_id, ordered by created_at	`idx_messages_conversation_created (conversation_id, created_at)`
app_registrations	List all enabled apps	`idx_apps_enabled (enabled) WHERE enabled = true`
user_app_tokens	Lookup by user_id + app_id	`UNIQUE (user_id, app_id)`
tool_invocations	Query by conversation_id for context; aggregate by app_id for analytics	`idx_invocations_conversation (conversation_id)`, `idx_invocations_app (app_id, created_at)`

Backup and disaster recovery: Railway provides daily automated backups for Postgres. For the demo, this is sufficient. In production, we would add point-in-time recovery and WAL archiving. The Grokipedia MCP server on Cloudflare is stateless, so no backup needed. OAuth tokens are the most critical data; loss means users must re-authorize (inconvenient but not catastrophic).

Phase 3: Post-Stack Refinement

12. Security & Sandboxing Deep Dive

iframe sandbox attributes: - Internal apps (Chess, Flashcards): `sandbox="allow-scripts allow-same-origin allow-forms"`. We control the code, so `allow-same-origin` is safe and enables localStorage access within the app. - External/untrusted apps (future): `sandbox="allow-scripts allow-forms"` only. No `allow-same-origin` prevents the iframe from accessing cookies or storage on the parent origin.

CSP headers for embedded content:

```
Content-Security-Policy:
  default-src 'self';
  frame-src https://chess.chatbridge.app https://flashcards.chatbridge.app;
  connect-src 'self' https://openrouter.ai;
  script-src 'self' 'unsafe-inline';
  style-src 'self' 'unsafe-inline';
```

Preventing apps from accessing parent DOM: The iframe `sandbox` attribute enforces this at the browser level. Even with `allow-scripts`, sandboxed iframes cannot access `window.parent.document`. The only communication channel is `postMessage`, which the platform validates by checking `event.origin` against the registered app's domain.

Rate limiting per app and per user: - Per-user: 60 chat messages/min, 120 API calls/min (Elysia rate limit plugin) - Per-app tool invocations: 30 calls/min per user per app (prevents a runaway LLM loop) - Grokipedia MCP: Cloudflare Workers handles its own rate limiting; we add a 100 req/min ceiling at the backend adapter level

13. Error Handling & Resilience

Failure Scenario	Detection	Recovery
iframe fails to load	`onError` event on iframe element; no `app:ready` within 5s	Show error card with retry button; LLM informed "app failed to load"
Tool call timeout	Backend sets 10s timeout on adapter `invokeTool()`	Return timeout error to LLM; LLM tells user "the app took too long to respond"
App crash mid-interaction	No heartbeat for 15s (heartbeat expected every 5s)	Show "app not responding" overlay with retry/dismiss; LLM informed of crash
LLM returns invalid tool call	Validate tool name and params against registered schemas	Return structured error to LLM for self-correction; log for monitoring
OAuth token refresh fails	HTTP 401 from Notion API after refresh attempt	Clear stored tokens; return `auth_required` to LLM; user prompted to re-authorize
Postgres connection failure	Drizzle connection error	Retry with exponential backoff (3 attempts); return 503 to client; Railway auto-restarts service
OpenRouter rate limit	HTTP 429 response	Retry after `Retry-After` header delay; show "AI is busy" indicator to user

Circuit breaker pattern for unreliable apps: If an app's tool invocations fail 5 times in a 5-minute window, the adapter marks it as `degraded`. In degraded state, the LLM system prompt includes "Note: [app_name] is currently experiencing issues" so it can inform the user rather than repeatedly attempting failed calls. The circuit resets after 5 minutes.

14. Testing Strategy

Plugin interface isolation testing: - Unit tests for each `PluginAdapter` implementation (MCPAdapter, RESTAdapter, IframeAdapter) using mocked external services - Test tool schema

validation: valid schemas pass, malformed schemas rejected - Test tool invocation routing: given a tool_call, verify the correct adapter is selected - Test timeout and error handling paths in each adapter

Mock apps for integration testing: - **Echo App**: minimal iframe app that mirrors every postMessage back. Used to validate the full postMessage protocol (init, tool invoke, state update, complete, heartbeat, error, resize) - Built as part of Phase 3 before the Chess app to prove the protocol works end-to-end

End-to-end testing of full invocation lifecycle: - Playwright tests covering the 7 testing scenarios from the spec: 1. User asks to use an app (tool discovery) 2. App UI renders in chat (iframe loading) 3. User interacts with app, returns to chat (completion signaling) 4. User asks about results after completion (context retention) 5. User switches between apps (multi-app) 6. Ambiguous query (routing) 7. Unrelated query (refusal)

Load testing with concurrent app sessions: - Not a priority for demo scale, but the architecture supports it: each user's app sessions are independent, adapters are stateless, and Postgres handles concurrent reads/writes via connection pooling

15. Developer Experience

How easy is it to build a third-party app? A developer needs to: 1. Create a web app (any framework) 2. Include the `chatbridge-sdk` package (~2KB, zero dependencies) 3. Call `ChatBridgeApp.init()` and register tool handlers 4. Deploy their app to any static hosting 5. Request registration on the ChatBridge platform (admin adds them to `app_registrations`)

Minimal example:

```
import { ChatBridgeApp } from '@chatbridge/sdk';

const app = ChatBridgeApp.init();

app.onToolInvoke('roll_dice', ({ sides }) => {
  return { result: Math.floor(Math.random() * sides) + 1 };
});

app.updateState({ lastRoll: null });
```

Documentation needed: - postMessage protocol specification (message types, expected flows) - chatbridge-sdk API reference (init, onToolInvoke, updateState, complete, error) - Tool schema format (JSON Schema for parameters) - Example apps (Echo App, Chess) as reference implementations - Security requirements (what sandbox restrictions apply, what APIs are available)

Local development workflow for app developers: - Run ChatBridge platform locally via `docker-compose up` (Postgres) + `bun dev` (API + frontend) - Add their app's localhost URL to

`app_registrations` via seed script or admin API - Their app runs on its own dev server; iframe loads from localhost - Hot reload works: changes to the app are reflected immediately in the iframe

Debugging tools for tool invocation failures: - `tool_invocations` table logs every call with input, output, status, and duration - Browser DevTools console shows all postMessage traffic (SDK logs in development mode) - Elysia backend logs adapter routing decisions and errors with structured JSON logging

16. Deployment & Operations

Where third-party apps get hosted: - Internal apps (Chess, Flashcards): deployed as static builds on Railway alongside the platform - MCP apps (Grokipedia): Cloudflare Workers - External SaaS apps (Notion): hosted by the provider; we only interact via API

CI/CD for the platform: - GitLab CI pipeline: lint (Biome) + type-check (tsc) + test (Vitest) + build - Railway auto-deploys from the main branch on push - Cloudflare Workers deployed via `wrangler deploy` in CI

Monitoring for app health and invocation success rates: - `tool_invocations` table provides queryable history of all tool calls with status and duration - Health check endpoint (`GET /api/health`) verifies Postgres connectivity and returns app registry status - Structured logging (JSON) in Elysia for error tracking - Stretch goal: dashboard querying `tool_invocations` for per-app success rates and p95 latencies

How to handle app updates without breaking existing sessions: - iframe apps load from a URL; updating the deployed static build takes effect on next iframe load - Active iframes are not affected mid-session (browser has already loaded the page) - MCP server updates on Cloudflare Workers are atomic (new code serves new requests; in-flight requests complete on old code) - Breaking tool schema changes require updating `app_registrations.config` in the database; active conversations using old schemas will see errors on their next tool call (acceptable for MVP; versioned schemas would be a future improvement)

Technical Details

System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│  Browser (Chatbox Fork - React SPA)                          │
│                                                              │
│   ┌──────────┐   ┌──────────┐   ┌──────────────────────┐    │
│   │ Chat UI  │   │ App      │   │ iframe (Chess /      │    │
│   │ (Chatbox)│   │ Registry │   │ Flashcards)          │    │
│   │          │◄──│ Panel    │   │   ▲                  │    │
│   └──────────┘   └──────────┘   │   │ postMessage      │    │
│        │                        │   ▼                  │    │
│        │                        │ ChatBridge SDK       │    │
│        │                        └──────────────────────┘    │
│        │ SSE (streaming)                                    │
└────────┼─────────────────────────────────────────────────────┘
         │ HTTPS
         ▼
┌──────────────────────────────────────────────────────────────┐
│  Elysia Backend (Bun)                                        │
│                                                              │
│   ┌──────────┐   ┌──────────┐   ┌──────────┐               │
│   │ Auth     │   │ LLM      │   │ Plugin   │               │
│   │(better-  │   │ Proxy    │   │ Registry │               │
│   │ auth)    │   │(OpenRouter│  │ + Router │               │
│   └──────────┘   └──────────┘   └──────────┘               │
│                                      │                      │
│   ┌──────────┐   ┌──────────┐       │                      │
│   │ OAuth    │   │ Chat     │   ┌───▼──────┐               │
│   │ Token    │   │ History  │   │ Adapters │               │
│   │ Vault    │   │ Store    │   │ ┌──────┐ │               │
│   └──────────┘   └──────────┘   │ │MCP   │ │               │
│                                 │ │REST  │ │               │
│                                 │ │iframe│ │               │
│   ┌──────────────────────┐      │ └──────┘ │               │
│   │  PostgreSQL          │      └──────────┘               │
│   │  - users, sessions   │           │                     │
│   │  - chat history      │           │                     │
│   │  - app registrations │      ┌────▼──────┐              │
│   │  - oauth tokens      │      │ External  │              │
│   │  - tool invocations  │      │ Services  │              │
│   └──────────────────────┘      │- Grokipedia MCP (CF)│    │
│                                 │- Notion API         │    │
│                                 └─────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

Data Flow: Tool Invocation

```
User message
  → Frontend sends to /api/chat/stream (with appContext map)
    → Elysia builds prompt with active tool schemas
      → OpenRouter LLM (function calling)
        → LLM returns tool_call
          → Plugin Router dispatches to adapter:
            ├─ MCPAdapter  → Streamable HTTP → Grokipedia CF Worker → result
            ├─ RESTAdapter → Notion API (with stored OAuth token) → result
            └─ IframeAdapter → returns render instruction to frontend
          → Result injected into LLM context
            → LLM streams final response
              → SSE → Browser renders response
```

iframe Dual Communication Path

```
Backend Path (tool invocations):
  LLM → tool_call → Elysia → "render chess iframe" → Frontend

Frontend Path (real-time state sync):
  iframe ←→ postMessage ←→ IframeBridge component ←→ appContext state
  (appContext sent with next /api/chat/stream request)
```

Tech Stack

Layer	Choice	Alternatives Considered	Why
Runtime	Bun	Node.js, Deno	Developer preference, fastest runtime, native TS
Backend	Elysia	Hono, Express	Developer preference, Bun-native, type-safe
Frontend	Chatbox fork (React 18, Vite, Zustand)	Next.js from scratch	Requirement to fork Chatbox
UI Libraries	Mantine + MUI (existing)	shadcn/ui	Already in Chatbox, no migration needed
Database	PostgreSQL + Drizzle	SQLite, Prisma	Concurrent access, Railway native, type-safe ORM
Auth	better-auth	Lucia, NextAuth, Clerk	Developer preference, modern, Bun-compatible
LLM	OpenRouter (Claude Sonnet 4)	Direct provider APIs	Multi-model routing, existing credits
AI SDK	Vercel AI SDK (existing)	LangChain	Already integrated in Chatbox
MCP Client	@modelcontextprotocol/sdk	Custom	Already in Chatbox, standard protocol

Layer	Choice	Alternatives Considered	Why
MCP Server	Custom TS on Cloudflare Workers	Python grokipedia-mcp	Stack uniformity, first-class CF support
Deployment	Railway + Cloudflare Workers	Vercel, Fly.io	Developer preference, Postgres included

Database Schema

```
-- Users & Auth (better-auth managed)
users (id, email, name, avatar_url, created_at)
sessions (id, user_id, expires_at, token)

-- Chat
conversations (id, user_id, title, created_at, updated_at)
messages (id, conversation_id, role, content, tool_calls jsonb, tool_call_id, metadat

-- Plugin System
app_registrations (id, name, description, type enum('iframe','mcp','rest'), auth_type

-- OAuth Token Vault
user_app_tokens (id, user_id, app_id, access_token encrypted, refresh_token encrypted

-- Observability
tool_invocations (id, conversation_id, message_id, app_id, tool_name, input jsonb, ou
```

Plugin Architecture Details

Adapter Interface:

```
interface PluginAdapter {
  type: 'iframe' | 'mcp' | 'rest';
  getTools(): Promise<ToolSchema[]>;
  invokeTool(name: string, params: Record<string, unknown>): Promise<ToolResult>;
  initialize(config: AppConfig, userTokens?: OAuthTokens): Promise<void>;
  shutdown(): Promise<void>;
}
```

Three implementations: - **MCPAdapter**: connects to remote MCP servers via Streamable HTTP (Grokipedia) - **RESTAdapter**: calls REST APIs with stored OAuth/API-key credentials (Notion) -

IframeAdapter: returns iframe render instructions; real-time comms handled by frontend (Chess, Flashcards)

postMessage Protocol:

```
// Platform → App
| { type: 'app:init'; sessionId: string; config: Record<string, unknown> }
| { type: 'tool:invoke'; id: string; tool: string; params: Record<string, unknown> }
| { type: 'app:destroy' }

// App → Platform
| { type: 'app:ready' }
| { type: 'tool:result'; id: string; result: unknown }
| { type: 'state:update'; state: Record<string, unknown> }
| { type: 'app:complete'; summary: string }
| { type: 'app:error'; error: string }
| { type: 'ui:resize'; height: number }
| { type: 'heartbeat' }
```

All messages validated against sender origin. Unknown origins silently dropped.

Completion Signaling:

1. App sends `{ type: 'app:complete', summary: "Game over. White won by checkmate in 24 moves." }`
2. Frontend dismisses iframe, injects summary into conversation as a system message
3. LLM can reference the summary in subsequent turns
4. For crashes: heartbeat (5s interval), “not responding” overlay after 15s of silence

Multi-App UX:

- One active iframe at a time
- Previous app interactions shown as collapsed “app cards” in message history
- Switching conversations unmounts iframe; returning re-initializes with last known state
- LLM system prompt: “If a user request could map to multiple apps, ask which they’d like to use”

Approaches Considered

Approach	Pros	Cons	Verdict
Extend Chatbox MCP only	Minimal changes, proven protocol	MCP has no UI rendering or completion signaling	Rejected: insufficient for iframe apps
Custom plugin system only	Full control, purpose-built	Doesn't leverage existing MCP infra	Rejected: reinvents the wheel for MCP-compatible services
Hybrid adapter pattern	Supports all integration types; MCP for MCP services, iframe for UI apps, REST for OAuth services	More adapter code to write	**Selected**
Backend-orchestrated WebSocket	Centralized, scalable	Over-engineered for demo scale; adds latency	Rejected: SSE + postMessage is simpler

---

Cost Analysis

Development Costs

Category	Calculation	Cost
OpenRouter LLM (dev)	~500 requests x ~2K tokens avg	~$3-5
Grokipedia API	Free, no key	$0
Notion API	Free tier	$0
Railway	Free trial / credits	$0
Cloudflare Workers	Free tier (100K req/day)	$0
Total		**~$5**

Production Cost Projections

Scale	Users	LLM Cost	Infrastructure	Total/mo	Assumptions
Small	100	$30	$10	~$40	5 sessions/user/mo, 10 msgs/session, ~1K tokens/msg
Medium	1,000	$300	$25	~$325	Same per-user patterns
Large	10,000	$3,000	$100	~$3,100	Heavier users, connection pooling
Scale	100,000	$25,000	$500	~$25,500	Model routing optimization, prompt caching

Assumptions: Claude Sonnet 4 via OpenRouter (~$3/1M input, ~$15/1M output). Average 1 tool invocation per 3 messages. Conversation context averages ~2K tokens.

Risks & Limitations

Risk	Severity	Mitigation
Chatbox codebase complexity	High	Focus on clean seams (Platform, RequestAdapter); don't refactor existing code
Completion signaling reliability	Medium	Timeout + heartbeat fallback; circuit breaker for repeat failures
Notion OAuth approval time	Low	Use internal integration (API key) for demo; OAuth for production
1-week timeline	High	Build vertically: Chess fully working before other apps
postMessage protocol design	High	Build echo-app test first to validate protocol before Chess

Explicitly NOT building: - Teacher admin dashboard - App marketplace / submission workflow - Multi-tenant organization support - Mobile version - Multi-user collaborative chat

---

Decision Log

#	Decision	Alternatives Considered	Rationale	Confidence
1	Fork Chatbox, modify web version	Build from scratch	Requirement; clean seams exist	Medium
2	Bun + Elysia backend	Hono, Express	Developer expertise, Bun-native	High
3	PostgreSQL + Drizzle	SQLite, Prisma	Concurrent access, Railway native	High
4	better-auth	Lucia, NextAuth	Developer preference, modern	High
5	Hybrid plugin adapter pattern	MCP-only, custom-only	Supports all integration types	High
6	iframe + postMessage for app UI	Web Components, Shadow DOM	True isolation, proven pattern	High
7	Cloudflare Workers for MCP server	Railway, co-hosted	First-class MCP support, free	High
8	Custom TS Grokipedia MCP	Use Python MCP	Stack uniformity	High
9	OpenRouter for LLM	Direct provider APIs	Multi-model, existing credits	High
10	One active iframe at a time	Multi-iframe	Simpler UX, avoids resource issues	Medium

#	Decision	Alternatives Considered	Rationale	Confidence
11	App state via postMessage to frontend to backend	Direct app-to-backend WebSocket	Simpler, no extra connection	High
12	Chess AI via LLM analysis	Stockfish integration	Simpler; adequate for K-12	Medium
13	SSE for chat streaming	WebSocket	Simpler, auto-reconnect, sufficient for unidirectional streaming	High
14	OAuth in popup (not iframe)	In-iframe OAuth	Providers block iframe OAuth via X-Frame-Options	High
15	Echo app before Chess	Jump straight to Chess	Validates protocol before building real app on it	High

---

App Integration Matrix

App	Pattern	Auth	Transport	Deployment	Tools
Chess	iframe + postMessage	None (Internal)	postMessage	Railway (static)	start_game, make_move, get_board_state, get_legal_moves, resign
Flashcards	iframe + postMessage	None (Internal)	postMessage	Railway (static)	create_deck, add_card, start_review, get_next_card, submit_answer, get_stats
Grokipedia	MCP (Streamable HTTP)	None (External Public)	HTTP	Cloudflare Workers	search, get_page, get_page_content, get_related_pages
Notion	REST + OAuth2	OAuth2 (External Authenticated)	HTTPS	Notion cloud (SaaS)	search_pages, create_page, append_to_page

Monorepo Structure

```
chatbridge/
├── apps/
│   ├── web/              # Chatbox fork (React SPA)
│   ├── api/              # Elysia backend (Bun)
│   ├── chess/            # Chess iframe app (React + chess.js)
│   └── flashcards/       # Flashcard iframe app (React + ts-fsrs)
├── packages/
│   ├── shared/           # Shared types, postMessage protocol
│   └── chatbridge-sdk/   # SDK for iframe apps to communicate with platform
├── mcp/
│   └── grokipedia/       # Grokipedia MCP server (Cloudflare Worker)
├── docker-compose.yml    # Local Postgres
├── presearch.md          # This document
├── PRD.md                # Phased implementation plan
└── package.json          # Bun workspace root
```